

Tag 4

Die Coding-Arena & Einstieg in Listen

Python-Grundlagen-Schulung

Was du heute lernst

1. Eine Stunde intensives **Üben** von allem bisher Gelernten
2. Was ist eine **Liste** – und warum brauche ich sie?
3. Wie **greife ich auf einzelne Werte** in einer Liste zu?
4. Wie **verändere** ich eine Liste – hinzufügen, entfernen, sortieren

Teil 1

Die Warm-up-Coding-Arena

Eine intensive Übungsstunde

Bevor es neuen Stoff gibt, festigst du erst einmal
alles, was du in den letzten drei Tagen gelernt hast:

Variablen • `input()` • Rechnen • Texte (Strings) • Fehlersuche

Ein Beispiel aus dem Warm-up

```
leitung_mbit = float(input("Bandbreite pro Leitung (Mbit/s): "))
anzahl = int(input("Anzahl der Leitungen: "))
gesamt = leitung_mbit * anzahl
print(f"Gesamtbandbreite: {gesamt} Mbit/s")
```

Nichts Neues hier – nur **input**, **Umwandlung** und **Rechnen**,
alles schon bekannt aus den letzten Tagen.

Auch dabei: kleine Fehlersuchen

```
# Fehlerhaft (User gibt 1.5 ein → Absturz):  
# datenmenge_gb = int(input("Wie viel GB? "))  
  
# Richtig:  
datenmenge_gb = float(input("Wie viel GB? "))
```

`int()` kann keine Dezimalzahl wie `1.5` verarbeiten.
`float()` schon!

10 Aufgaben warten auf dich

Von Bandbreiten-Rechnern über String-Slicing
bis zu kleinen Fehlersuchen –
alles im Colab-Notebook.

Danach geht es mit einem großen neuen Thema weiter: **Listen!**

Teil 2

Was ist eine Liste?

Das Problem: viele einzelne Variablen

```
hostname1 = "SRV-01"  
hostname2 = "SRV-02"  
hostname3 = "SRV-03"
```

Das geht bei 3 Servern noch.

Aber was, wenn du **50, 500 oder 5000** Server
verwalten musst? 😞

Die Lösung: eine Liste

```
server_liste = ["SRV-01", "SRV-02", "SRV-03"]
```

Ein Name – aber beliebig viele Werte darin!

Eckige Klammern [] markieren Anfang und Ende,
Kommas trennen die einzelnen Werte.

Bild: Der Aktenschrank

Stell dir einen **Aktenschrank mit nummerierten Schubladen** vor.

Statt für jedes Dokument einen eigenen Tisch (= eine eigene Variable) zu haben, packst du alles in **einen Schrank**.

Jede Schublade hat eine Nummer –
du kannst gezielt eine öffnen, ohne die anderen anzufassen.

Warum ist das in der IT so wichtig?

Eine Liste aller aktiven Benutzer.

Eine Liste aller offenen Tickets.

Eine Liste aller IP-Adressen im Netzwerk.

Fast **jedes echte Programm** muss irgendwann mehrere Dinge gleichzeitig verwalten.

Auf eine Schublade zugreifen

Der Index

Jedes Element hat eine Nummer

Liste:	"192.168.1.1"	"192.168.1.2"	"192.168.1.3"
Index:	0	1	2

⚠️ Ganz wichtig: Python zählt **bei 0**, nicht bei 1!

Das **erste** Element hat den Index **0**.

Zugriff auf ein Element

```
ips = ["192.168.1.1", "192.168.1.2", "192.168.1.3"]  
  
print(ips[0])    # Erstes Element  
print(ips[-1])   # Letztes Element  
print(len(ips))  # Wie viele Elemente sind drin?
```

`-1` heißt "von hinten gezählt, das erste" –
egal wie lang die Liste ist!

`len(...)` zählt dir die Elemente.

Eine Liste verändern

Der große Unterschied zu Strings

Erinnerung: Strings kann man nicht ändern

Bei einem String konntest du **kein einzelnes Zeichen** nachträglich austauschen.

Du musstest immer einen **komplett neuen** String bauen
(z. B. mit `.replace()`).

Listen sind anders: sie sind veränderbar!

```
hardware = ["Maus", "Tastatur", "Monitor"]  
hardware[1] = "Headset"  
print(hardware)
```

```
['Maus', 'Headset', 'Monitor']
```

Du kannst **direkt** ein Element austauschen –
wie den Inhalt einer Schublade wechseln,
ohne den ganzen Schrank neu zu bauen.

Merksatz

Strings: unveränderbar

Listen: veränderbar



Kurze Pause

Teil 3

Listen wachsen & schrumpfen lassen

Element hinzufügen: .append()

```
tools = ["Ping", "Nslookup"]  
tools.append("Tracert")  
print(tools)  # ['Ping', 'Nslookup', 'Tracert']
```

`.append(...)` hängt einen Wert **ganz hinten** an.

Die häufigste Listen-Methode überhaupt!

Element gezielt einfügen: .insert()

```
tools.insert(1, "Nmap")  
print(tools)  # ['Ping', 'Nmap', 'Nslookup', 'Tracert']
```

`.insert(position, wert)` schiebt einen Wert an eine **bestimmte Stelle** – alles danach rutscht weiter.

Bild: die Warteschlange am Schalter

`.append(...)` = ein neuer Kunde stellt sich **hinten** an

`.insert(0, ...)` = ein dringender Fall wird
ganz vorne eingelassen – alle anderen rücken auf

Element entfernen (nach Wert): .remove()

```
tools = ["Ping", "Nmap", "Nslookup"]  
tools.remove("Ping")  
print(tools)  # ['Nmap', 'Nslookup']
```

`.remove(wert)` sucht diesen **Wert** in der Liste
und löscht ihn.

⚠ Ist der Wert nicht enthalten → Fehler (`ValueError`)!

Element entfernen (nach Position): .pop()

```
tools = ["Nmap", "Nslookup"]  
entfernt = tools.pop(0)  
print("Entfernt:", entfernt)  
print("Verbleibend:", tools)
```

`.pop(position)` entfernt das Element an dieser Stelle
und gibt es dir gleich zurück – zum Weiterverwenden!

Merkregel: remove vs. pop

`.remove()` → "Ich weiß, **was** weg soll"

(ich kenne den Wert, nicht die Position)

`.pop()` → "Ich weiß, **wo** es steht"

(und ich will den Wert noch weiterverwenden)

Teil 4

Jetzt du: der große Listen-Praxis-Slam

Was dich erwartet

Teil A – IT-Inventar & Daten sammeln

Server-Fuhrpark, IP-Erfassung, Gruppen-Bereinigung, Backups korrigieren

Teil B – ToDo-Listen & Warteschlangen

Tagesplanung, Ticket-Warteschlange, sortierte Lieferung, Deinstallation

Alle Aufgaben und Lösungen findest du im **Colab-Notebook**.

Ein Beispiel zum Warmwerden

```
admins = ["claudia", "stefan", "andreas", "markus"]  
admins.append("julia")  
admins.remove("stefan")  
print(admins)
```

Eine ganz normale Kombination aus
.append() und .remove() – genau das übst du jetzt selbst!

Zum Schluss

Ein typischer Fehler

IndexError: list index out of range

```
liste = ["A", "B", "C"]  
print(liste[10])
```

✗ Diese Liste hat nur **3** Elemente (Index 0, 1, 2) –
Index **10** gibt es gar nicht!

Bei diesem Fehler: einfach nochmal mit `len(...)` prüfen,
wie groß die Liste wirklich ist.

Was du heute gelernt hast

- ✓ Eine **Liste** speichert mehrere Werte unter einem Namen
- ✓ Zugriff über den **Index**, beginnend bei 0
- ✓ Listen sind **veränderbar** – anders als Strings
- ✓ `.append()` , `.insert()` , `.remove()` , `.pop()` verändern die Liste

Morgen: der große Praxis-Marathon 🎉